
Aprendizaje Automático 1

Práctica 1: Aprendizaje Supervisado

GRUPO 7

Bedoya Piedrahita, Brian

Santiago Ruiz, David

Curso 2025-26

Grado en Ciencia de Datos e Inteligencia Artificial
(ETSISI - UPM)

Índice

1. Introducción	3
2. Preprocesado de datos	4
2.1. Análisis Exploratorio y Calidad de Datos	4
2.2. División Estratificada de Datos (Prevención de <i>Data Leakage</i>)	4
2.3. Imputación de Valores Nulos	5
2.4. Codificación y Escalado	5
3. Selección de Características	6
3.1. Método 1: Filtro (f_classif)	6
3.2. Método 2: Wrapper (RFE)	7
3.3. Método 3: Embebido (Lasso L1)	7
4. Resultados y Selección del Pipeline Final	7
4.1. Comparativa de Rendimiento (F1-Score)	7
4.2. Decisión del Pipeline y Conjunto de Features	8
4.3. Análisis Visual de los Pipelines (V1 vs. V2)	8
5. Aplicación de Modelos de Aprendizaje Supervisado	10
5.1. Carga y División de Datos	10
5.2. Metodología de Evaluación	10
5.3. Búsqueda de Modelos	11
5.3.1. Modelos Lineales (Logística, Polinomial y SGD)	11
5.3.2. Árboles de Decisión y Ensamblados	11
5.3.3. Aprendizaje Basado en Instancias (KNN)	11
5.3.4. Modelos de Gradient Boosting (GBM)	12
5.3.5. Máquinas de Vectores de Soporte (SVM / SVC)	12
5.3.6. Red Neuronal	12
5.4. Análisis Comparativo	13
5.5. Optimización Exhaustiva (GridSearchCV)	14
5.6. Conclusión y Modelo Final	14
6. Predicciones Finales	15
6.1. Carga y Preparación de Datos	15
6.2. Replicación del Pipeline V2	16
6.2.1. Imputación de Valores Nulos	16
6.2.2. Transformación y Selección de Características	16
6.3. Selección del Modelo Final	16
6.4. Entrenamiento y Generación de Predicciones	16

1. Introducción

Esta práctica tiene como objetivo poner en práctica los conceptos y técnicas de aprendizaje automático estudiados en la asignatura. Se busca poner en práctica un flujo de trabajo completo que abarca desde el **preprocesado de datos** hasta la implementación y evaluación de **modelos de aprendizaje supervisado**.

El problema de clasificación seleccionado para este fin consiste en predecir si un individuo gana más o menos de 50.000\$ anuales, basándose en un conjunto de 13 características demográficas y laborales. El dataset de entrenamiento (`salario.csv`) consta de 27.998 instancias, mientras que un conjunto de test (`salario_test.csv`) se reserva para la evaluación final en una competición de Kaggle.

La métrica principal de evaluación es el **F1-Score**, una elección interesante dada la naturaleza del dataset, que presenta un desbalanceo de clases significativo (aproximadamente 76 % para la clase negativa y 24 % para la positiva).

Esta memoria detalla el trabajo realizado, dividiéndolo en los apartados que se corresponden con el flujo de trabajo ejecutado en los tres *notebooks* del proyecto:

1. **Preprocesado de Datos.** Detalla el análisis de calidad de los datos (identificando '?' como nulos), la división 80/20 estratificada para prevenir *data leakage*, y la imputación por moda. También se definen los dos *pipelines* experimentales: **V1** (con `RobustScaler` y `SMOTE`) y **V2** (con `StandardScaler` y ponderación de clases).
2. **Selección de Características.** Explica la aplicación de las tres familias de métodos de selección (Filtro `f_classif`, Wrapper RFE y Embebido L1-Lasso) a ambos *pipelines* (V1 y V2) para reducir la dimensionalidad de 87 a 30 *features*.
3. **Resultados y Selección del Pipeline Final.** Compara empíricamente el rendimiento de V1 frente a V2 usando una `LogisticRegression` como 'juez'. Se demuestra la superioridad del Pipeline V2 (Ponderado) y se justifica la selección del conjunto de '**Consenso V2 (24 features)**' como el dataset de entrada (`processed_dataset.csv.zip`) para la fase de modelado.
4. **Aplicación de Modelos de Aprendizaje Supervisado.** Utilizando el dataset de 24 *features*, se realiza una búsqueda *baseline* de 9 familias de modelos (Lineal, Polinomial, SGD, Árboles, Random Forest, KNN, SVM, GBMs y Keras). Se detalla la optimización del umbral de Keras (a 0.69) y la optimización exhaustiva (`GridSearchCV`) sobre los modelos 'ganadores', identificando a **XGBoost (F1: 0.7230)** como el ganador en nuestra validación local.
5. **Predicciones Finales.** En referencia al *notebook* de predicciones reales (`best_model.ipynb`). Este apartado explica la decisión final de usar **Keras** (F1 de Kaggle: 0.861) sobre XGBoost (F1 local: 0.7230) debido a su mejor generalización en datos externos. Finalmente, detalla el re-entrenamiento de Keras sobre el 100 % de los datos y la generación del archivo

`test_labels.csv` para la competición.

2. Preprocesado de datos

Nuestro enfoque se basó en un pipeline experimental, comparando dos estrategias principales (V1 y V2) para el manejo de outliers y desbalanceo de clases.

2.1. Análisis Exploratorio y Calidad de Datos

La carga inicial del dataset (`salario.csv`) y un análisis preliminar con `.info()` mostraron 13 columnas y 27.998 instancias, sin valores nulos aparentes. Sin embargo, un análisis más profundo de las distribuciones de valores (`.value_counts()`) reveló un problema de calidad de datos: la presencia del carácter '?' como marcador de datos faltantes en las columnas categóricas `trabajo`, `trabajo.1` y `pais-origen`.

```
trabajo.1
Prof-specialty    3566
Craft-repair      3512
Exec-managerial   3464
Adm-clerical      3305
Sales             3148
Other-service     2857
Machine-op-inspct 1712
?                 1573
Transport-moving  1372
Handlers-cleaners 1146
Farming-fishing   859
Tech-support      791
Protective-serv   552
Priv-house-serv   133
Armed-Forces       8
Name: count, dtype: int64
```

Figura 1: Salida de `value_counts()` para la columna 'trabajo.1', mostrando 1573 instancias con el marcador '?'.

Al ser un string, Pandas no lo reconocía como nulo, lo que habría causado errores en etapas posteriores. Lo solucionamos recargando el dataset y especificando `na_values=['?']`, permitiendo que Pandas los identificara correctamente como NaN (Not a Number).

2.2. División Estratificada de Datos (Prevención de *Data Leakage*)

Antes de aplicar cualquier transformación, se dividió el dataset en conjuntos de entrenamiento (80 %) y validación (20 %).

Este es el paso más importante para garantizar la validez de nuestro modelo. Cualquier operación de preprocesado que aprenda parámetros de los datos (como la media para escalar o la moda para imputar) debe realizarse después de la división y debe aprender únicamente del conjunto de entrenamiento. Si se realizara antes, el modelo de preprocesado 'vería' los datos de validación,

introduciendo información del futuro en el entrenamiento. Esto llevaría a métricas de rendimiento infladas y poco realistas.

Con un dataset de 28.000 muestras, un conjunto de validación del 20 % (5.600 muestras) es estadísticamente robusto para una evaluación fiable. Esta proporción nos permite maximizar la cantidad de datos para el entrenamiento (80 % o 22.398 muestras), permitiendo a los modelos aprender patrones complejos sin sacrificar la confianza en su evaluación.

Utilizamos `stratify=y` debido al desbalanceo de clases (76 % para ' $\leq 50K$ ' y 24 % para ' $> 50K$ '). La estratificación asegura que esta proporción se mantenga idéntica en ambos conjuntos, garantizando que el modelo se entrene y evalúe en condiciones representativas.

2.3. Imputación de Valores Nulos

Tras la división, las columnas `trabajo`, `trabajo.1` y `pais-origen` presentaban un número considerable de valores NaN.

Análisis de Alternativas:

- *Eliminación de filas* : Se descartó esta opción. La columna `trabajo`, por ejemplo, tenía 1.249 nulos en el conjunto de entrenamiento. Eliminarlos habría supuesto una pérdida de más del 5.5 % de los datos de entrenamiento, reduciendo la capacidad del modelo para aprender.
- *Crear una categoría 'Desconocido'*: Esta es una técnica válida, pero introduce una nueva dimensión que el modelo debe interpretar.
- *Imputación por Moda*: Se eligió esta técnica por su simplicidad y eficacia en variables categóricas.

Implementación: Calculamos la moda para cada una de las tres columnas usando únicamente el conjunto `X_train`. Posteriormente, se usaron estas modas para rellenar los valores NaN tanto en `X_train` como en `X_val`. Esto asegura que no haya fuga de datos desde la validación hacia el entrenamiento.

2.4. Codificación y Escalado

El núcleo del preprocesado consistió en un experimento comparativo entre dos pipelines (V1 y V2) para determinar la mejor estrategia de transformación y manejo del desbalanceo de clases. Se utilizó un `ColumnTransformer` para aplicar tratamientos específicos a cada subconjunto de variables.

En ambos pipelines, las variables categóricas recibieron una codificación diferenciada:

- **Ordinales ('estudios')**: Se aplicó `OrdinalEncoder` utilizando un orden jerárquico explícito (de '`Preschool`' a '`Doctorate`') para preservar su valor informativo.
- **Binarias ('sexo')**: Se codificaron a 0 y 1 usando `OrdinalEncoder`.

- **Nominales** (ej. 'trabajo', 'estado-civil'): Las columnas sin orden inherente fueron transformadas con `OneHotEncoder`, creando nuevas columnas *dummy*.

Donde se distingue entre ambos enfoques es en el tratamiento de las características numéricas ('edad', 'ganancias_inversiones', 'perdidas_inversiones' y 'horas-trabajo_semana'), que presentaban alta asimetría y valores atípicos extremos (ej. 99999).

Pipeline V1 (RobustScaler + SMOTE): Esta estrategia utilizó `RobustScaler`, un escalador basado en cuantiles que es inherentemente robusto a los valores atípicos, evitando que estos distorsionen la escala. Para combatir el desbalanceo de clases (76 % vs 24 %), se aplicó `SMOTE` (Synthetic Minority Over-sampling Technique) al conjunto de entrenamiento.

Pipeline V2 (StandardScaler + Ponderación): Esta estrategia utilizó `StandardScaler`, el cual es sensible a los outliers, para probar si el propio modelo podía gestionar la distorsión. El desbalanceo se manejó directamente en el estimador (ej. Regresión Logística) usando el parámetro `class_weight='balanced'`, que ajusta las penalizaciones por error en la clase minoritaria.

El motivo de esta bifurcación en dos enfoques fue determinar la estrategia óptima para gestionar simultáneamente los outliers extremos y el fuerte desbalanceo de clases. El **Enfoque V1 (Intervención de Datos)** probó si el modelo rendiría mejor con datos 'limpios', usando `RobustScaler` (robusto a outliers) y `SMOTE` (para balancear el dataset). En contraposición, el **Enfoque V2 (Adaptación del Modelo)** probó si era más efectivo usar un `StandardScaler` (sensible a outliers) y forzar al propio algoritmo a gestionar el desbalanceo mediante la ponderación de clases (`class_weight='balanced'`). En esencia, comparamos si era mejor intervenir fuertemente los datos (V1) o dejar que el modelo compensara las imperfecciones estadísticas (V2).

3. Selección de Características

Con 87 características tras el OHE, la selección de características es vital para reducir el ruido, evitar la maldición de la dimensionalidad y mejorar la interpretabilidad. Se realizó una evaluación exhaustiva de los tres métodos vistos en la asignatura, aplicando cada uno a ambos pipelines (V1 y V2). El objetivo fue reducir la dimensionalidad a un valor heurístico de $k = 30$.

3.1. Método 1: Filtro (`f_classif`)

Los métodos de filtro evalúan cada característica de forma univariante, independientemente del modelo.

- **Técnica:** `SelectKBest` con `f_classif` (Test ANOVA F-value).
- **Justificación:** Se eligió `f_classif` por ser ideal para nuestro escenario: un target binario (salario) y características de entrada numéricas (resultado de nuestro preprocesado). Mide si la diferencia en las medias de una característica entre las dos clases es estadísticamente significativa. Es computacionalmente muy rápido.

3.2. Método 2: Wrapper (RFE)

Los métodos wrapper utilizan un modelo para evaluar subconjuntos de características.

- **Técnica:** RFE (Recursive Feature Elimination) con un estimador `LogisticRegression`.
- **Justificación:** RFE se eligió sobre otros métodos wrapper (como *Forward Selection*) por su eficacia. Entrena un modelo, extrae la importancia de las características (los `.coef_`) y elimina recursivamente las menos importantes hasta alcanzar el k deseado. Esto le permite capturar interacciones entre variables (a diferencia de los filtros) de una forma más eficiente que una búsqueda exhaustiva. Se usó Regresión Logística por su velocidad y su capacidad de proveer coeficientes de importancia.

3.3. Método 3: Embebido (Lasso L1)

Los métodos embebidos integran la selección de características en el propio entrenamiento del modelo.

- **Técnica:** `SelectFromModel` con `LogisticRegression` y penalización `penalty='l1'` (LASSO).
- **Justificación:** La regularización L1 (LASSO) añade a la función de coste una penalización proporcional al valor absoluto de los coeficientes. Esto tiene el efecto de encoger los coeficientes de las características irrelevantes a **exactamente cero**, realizando así una selección automática. Se prefirió sobre L2 (Ridge), que solo reduce los coeficientes pero nunca los elimina.

4. Resultados y Selección del Pipeline Final

4.1. Comparativa de Rendimiento (F1-Score)

Para decidir qué pipeline y qué conjunto de features era superior, se entrenó un modelo 'juez', el cual en este caso fue `LogisticRegression` en cada uno de los 6 conjuntos de datos de entrenamiento y se evaluó el F1-score en el conjunto de validación. Se incluyeron también los 'Baseline' (usando las 87 features) y dos 'consensos' (features seleccionadas por al menos 2 métodos, y features seleccionadas por los 3).

Cuadro 1: Comparativa de F1-Score entre Pipelines (Modelo Juez: LogReg)

Método de Selección	F1 - Pipeline V1 (SMOTE)	F1 - Pipeline V2 (Ponderado)
Baseline (87 features)	0.6828	0.6827
Filtro (30 features)	0.6698	0.6795
Wrapper (30 features)	0.6265	0.6458
Embebido (30 features)	0.6310	0.6841
Consenso 'Al menos en 2'	0.6312	0.6798
Consenso 'En los 3'	0.5731	0.6472

Interpretación de Resultados: Los resultados de la Tabla 1 son claros:

1. El **Pipeline V2 (Ponderado)** superó sistemáticamente al **Pipeline V1 (SMOTE)** en todos los métodos de selección de características.
2. El método **Embebido (L1) V2** fue el único que logró superar al baseline, demostrando la eficacia de la regularización L1 tanto para modelar como para seleccionar.
3. El rendimiento de V1 fue consistentemente mediocre. Esto sugiere fuertemente que **SMOTE introdujo ruido** o patrones sintéticos que confundieron al modelo lineal, perjudicando su capacidad de generalización en el conjunto de validación.
4. El éxito de V2 demuestra que la estrategia pragmática fue la ganadora: el modelo de Regresión Logística (con su propia regularización y `class_weight`) fue capaz de manejar las features distorsionadas por los outliers (vía 'StandardScaler') mejor de lo que pudo manejar los datos sintéticos de SMOTE.

4.2. Decisión del Pipeline y Conjunto de Features

Basado en esta evidencia empírica, se tomó la decisión de **descartar el Pipeline V1 y proceder únicamente con el Pipeline V2**.

Como conjunto final de características, se seleccionó el '**Consenso “Al menos 2” V2**'. Aunque el método 'Embebido V2' obtuvo un F1-score 0.0043 superior, la diferencia es mínima. Se prefirió el conjunto de 'Consenso' (24 features) por ser teóricamente más robusto, ya que representa un acuerdo entre múltiples filosofías de selección. Este pequeño 'trade-off' en el F1-score se considera aceptable a cambio de una mayor estabilidad y generalización del conjunto de features.

El resultado de este apartado es el fichero `processed_dataset.csv.zip`. Este dataset, basado en el Pipeline V2 y el conjunto de features de 'Consenso', contiene 27.998 filas y 25 columnas (las 24 características seleccionadas + 1 variable objetivo). Este será el único dataset de entrada para el Apartado 2.

4.3. Análisis Visual de los Pipelines (V1 vs. V2)

Una comparación visual de la importancia de características, obtenida del modelo juez sobre los conjuntos de datos finales de cada pipeline, confirma esta decisión.

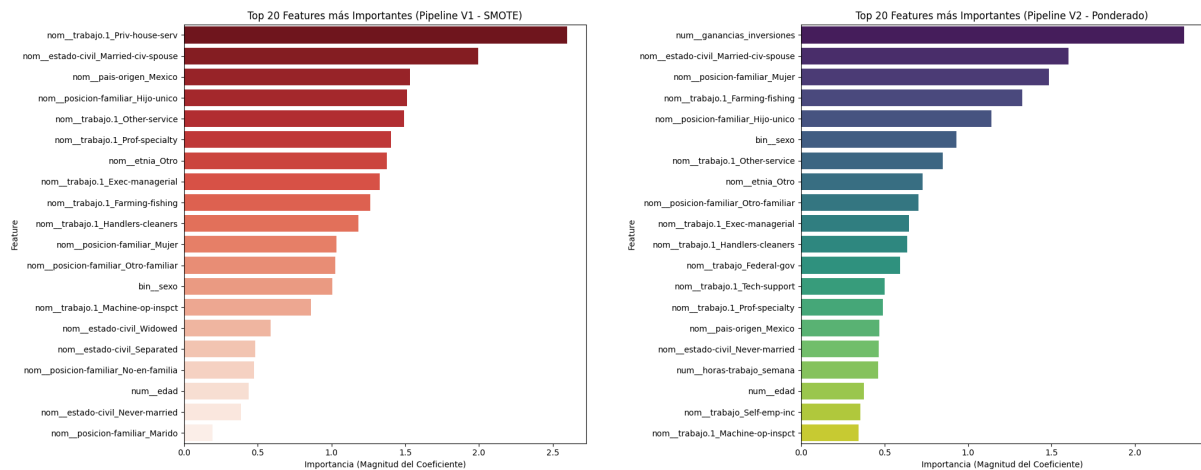


Figura 2: Comparativa de Importancia de Características: V1 (SMOTE, izq.) vs. V2 (Ponderado, der.). El V2 (derecha) asigna una alta importancia a variables numéricas clave como **edad** y **ganancias_inversiones**. En contraste, el V1 (izquierda), influenciado por SMOTE, prioriza categorías nominales (como **posicion-familiar_Marido**). Esto demuestra que el V2 identificó predictores más intuitivos y robustos.

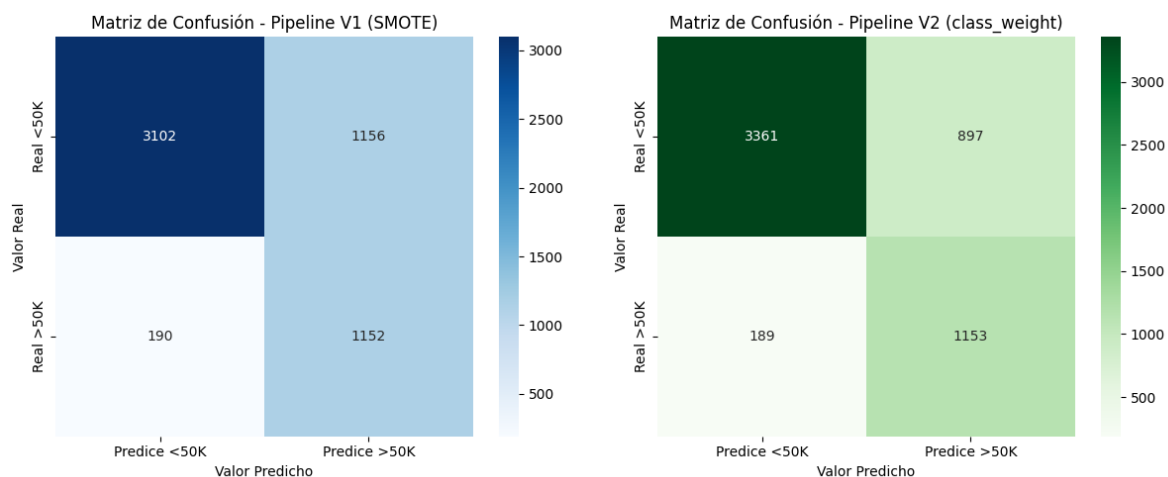


Figura 3: Matrices de Confusión. V1 (SMOTE, izq.) vs. V2 (Ponderado, der.).

La **Figura 3** es la justificación visual más contundente. La matriz de V1 (izquierda) muestra un número muy elevado de **Falsos Positivos (897)**. Es decir, el modelo con SMOTE predijo incorrectamente que 897 personas ganaban >50K.

En contraste, la matriz de V2 (derecha) reduce drásticamente este error a solo **189 Falsos Positivos**. Aunque V2 identifica ligeramente menos Verdaderos Positivos (1153 vs 1152 de V1), la reducción masiva de Falsos Positivos resulta en una Precisión mucho mayor y, consecuentemente, un F1-Score superior, validando nuestra elección.

5. Aplicación de Modelos de Aprendizaje Supervisado

Tras la fase de preprocesado y selección de características (Apartado 1), el objetivo de esta sección es aplicar, evaluar y optimizar rigurosamente un amplio espectro de algoritmos de clasificación. El objetivo final es seleccionar el modelo con el F1-Score (Clase 1) más alto para la predicción de salarios en este dataset.

5.1. Carga y División de Datos

El punto de partida de esta fase fue el conjunto de datos `processed_dataset.csv.zip`. Este archivo, generado en el Apartado 1, contiene las 24 características identificadas por la estrategia ganadora "V2 (Ponderado) + Consenso ≥ 2 ".

Para asegurar una evaluación final 100 % imparcial y coherente con el preprocesado, no se realizó una nueva división aleatoria. En su lugar, el dataset de 27,998 filas se cargó y se dividió manualmente usando los mismos índices de la división 80/20 original:

- **Conjunto de Entrenamiento (X_{train} , y_{train}):** Las primeras 22,398 filas (80 %). Este conjunto se utilizó para la validación cruzada y la búsqueda de hiperparámetros.
- **Conjunto de Prueba (X_{test} , y_{test}):** Las últimas 5,600 filas (20 %). Este conjunto se reservó exclusivamente para la evaluación final y la generación de métricas de los modelos ya optimizados.

5.2. Metodología de Evaluación

Para garantizar una comparación justa y robusta entre los distintos modelos, se definió un protocolo de evaluación que se aplicaría a todas las búsquedas de hiperparámetros.

1. **Métrica de Optimización (F1-Score):** Dado que el objetivo es identificar a los individuos con altos ingresos (la Clase 1, que representa solo el 24 % de los datos), la precisión (*accuracy*) es una métrica engañosa. La métrica seleccionada fue el **F1-Score de la clase positiva**, para buscar el equilibrio óptimo entre la Precisión y el Recall en la clase minoritaria, que es la más difícil de predecir.
2. **Gestión del Desbalance (Ponderación):** Como se determinó en el Apartado 1, la estrategia ganadora (V2) consistió en gestionar el desbalanceo a nivel de modelo. Se evitó el sobremuestreo (SMOTE) —que puede introducir ruido y patrones artificiales— y, en su lugar, se utilizó la **ponderación de clases** (ej. `class_weight='balanced'`). Esto ajusta la función de coste del algoritmo para que penalice mucho más los errores cometidos en la Clase 1, forzando al modelo a prestarle atención sin alterar los datos de entrenamiento.
3. **Validación Cruzada Estratificada:** Dado que el conjunto `y_train` sigue estando desbalanceado, una validación cruzada estándar ('KFold') podría generar *folds* con proporciones de clase erráticas. Para evitarlo, se utilizó `StratifiedKFold(n_splits=5)`, asegurando que cada uno de los 5 *folds* de validación mantuviera la proporción original de 76 %/24 %.

Para ejecutar estas evaluaciones de forma sistemática, se implementó una función *wrapper* ('run_gridsearch') que encapsula el proceso de `GridSearchCV`, el entrenamiento, la evaluación en `X_test` y la generación de reportes de clasificación y matrices de confusión.

5.3. Búsqueda de Modelos

El primer paso fue realizar una búsqueda `GridSearchCV` con conjuntos de parámetros ligeros para 9 familias de modelos. El objetivo era establecer un F1-score base e identificar los algoritmos más prometedores.

5.3.1. Modelos Lineales (Logística, Polinomial y SGD)

Estos modelos sirvieron para establecer el rendimiento base y comprender la linealidad del problema.

- **Regresión Logística:** Actuó como nuestro *baseline* fundamental. Al ser un modelo lineal, intenta trazar un hiperplano para separar las clases. Su F1-score (**0.6797**) fue nuestra puntuación inicial a batir.
- **Regresión Polinomial:** Al incluir interacciones de Grado 2 (ej. `edad * horas-trabajo`), el F1-score mejoró notablemente a **0.6929**. Esta fue la primera evidencia clave de que el problema no es puramente lineal y que las interacciones entre características son importantes.
- **SGD Classifier:** Este modelo arrojó un F1-score (0.6876) casi idéntico al de la Regresión Logística. Esto es lógico, ya que al seleccionar `loss='log_loss'`, el `SGDClassifier` está, en esencia, entrenando el mismo modelo.

Los modelos lineales demuestran un 'techo' de rendimiento claro en torno a 0.69.

5.3.2. Árboles de Decisión y Ensamblados

- **Árbol de Decisión:** Un único árbol (optimizado a `max_depth=10`) superó ligeramente a la regresión logística (F1: 0.6828), confirmando de nuevo la naturaleza no-lineal de los datos.
- **Random Forest:** Este modelo de ensamblado (*Bagging*) demostró un salto de rendimiento significativo, alcanzando un F1-Score de **0.7062**. Al promediar cientos de árboles (cada uno entrenado en una sub-muestra de datos y características), reduce drásticamente el sobreajuste del árbol individual y captura patrones no-lineales más robustos.

5.3.3. Aprendizaje Basado en Instancias (KNN)

K-Nearest Neighbors (KNN) obtuvo el peor resultado de todos los modelos (F1: 0.6462). Esto era esperado, ya que el modelo sufre por dos razones: no soporta la ponderación de clases (quedando sesgado por la clase mayoritaria) y su concepto de 'distancia' pierde fiabilidad en un espacio de 24 dimensiones ('maldición de la dimensionalidad').

5.3.4. Modelos de Gradient Boosting (GBM)

Esta familia de algoritmos, que aprende de forma secuencial corrigiendo los errores de los árboles anteriores, mostró una superioridad inmediata sobre todos los demás:

- **XGBoost:** F1-Score = 0.7194
- **LightGBM:** F1-Score = 0.7210
- **CatBoost:** F1-Score = 0.7182

Los tres modelos de *boosting* superaron la barrera del 0.718, posicionándose como los claros candidatos para la optimización exhaustiva.

5.3.5. Máquinas de Vectores de Soporte (SVM / SVC)

Para esta familia, la búsqueda se dividió estratégicamente en dos partes para utilizar los estimadores más eficientes (**LinearSVC** y **SVC**), ambos con `class_weight='balanced'`.

- **Kernel Lineal:** Se utilizó **LinearSVC** por su rapidez, pero obtuvo un F1-Score de **0.6758**. Este resultado, inferior al *baseline* de Regresión Logística, mostró un pobre equilibrio Precisión/Recall (0.55 / 0.87) debido a la ponderación de clases.
- **Kernels No-Lineales:** Se utilizó **SVC** para probar 'rbf' y 'poly'. Esta búsqueda seleccionó el kernel polinómico como el óptimo, mejorando el F1-Score a **0.6849**.

Aunque el kernel polinómico superó al lineal, el rendimiento general fue inferior al de los modelos de ensamblado (Random Forest/Boosting).

5.3.6. Red Neuronal

Finalmente, se entrenó una red neuronal (MLP) con una arquitectura de 'embudo' (**Input(24) ->Dense(64) ->Dropout(0.3) ->Dense(32) ->Dropout(0.3) ->Dense(1)**), diseñada para capturar interacciones no lineales complejas.

Al analizar los resultados iniciales, observamos que el uso de `class_weight='balanced'` logró un excelente Recall (0.87), pero a costa de una pobre Precisión (0.58), resultando en un F1-Score subóptimo de 0.6940.

Para corregirlo, se realizó una **Optimización del Umbral de Decisión** (*Threshold Tuning*). Se evaluó el F1-Score en un rango de umbrales (ver Figura 4). Se descubrió que el umbral óptimo no era 0.5, sino **0.69**. Al exigir al modelo una confianza del 69% antes de predecir »50K", el F1-Score final de la red se disparó a **0.7109**.

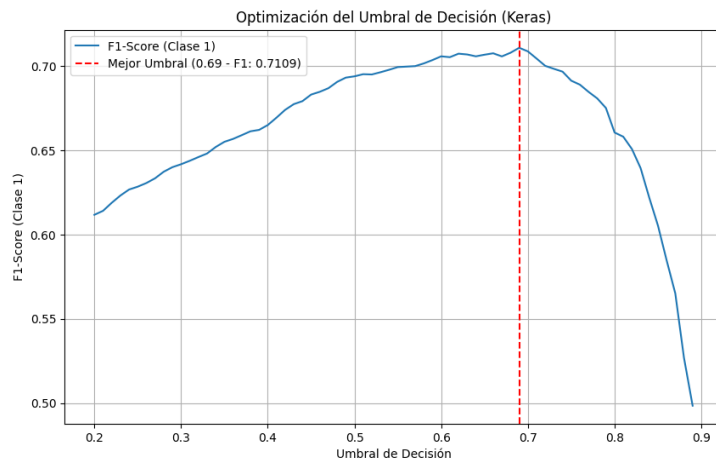


Figura 4: Búsqueda del Umbral de Decisión Óptimo para Keras. El F1-Score (eje Y) se maximiza en un umbral (eje X) de 0.69.

5.4. Análisis Comparativo

La búsqueda *baseline*, resumida en la Tabla 1 y visualizada en la Figura 5, revela una jerarquía de rendimiento clara. Los modelos lineales (Logística, SGD) marcan un 'techo lineal' de ~ 0.68 . La Regresión Polinomial y los modelos de ensamblado (Random Forest, Keras) rompen esta barrera y alcanzan un F1 de ~ 0.70 , confirmando la naturaleza no-lineal del problema.

Finalmente, muestra que los tres algoritmos de **Gradient Boosting** (LGBM, XGBoost, CatBoost) superan a todos los demás, estableciéndose por encima de **0.718**.

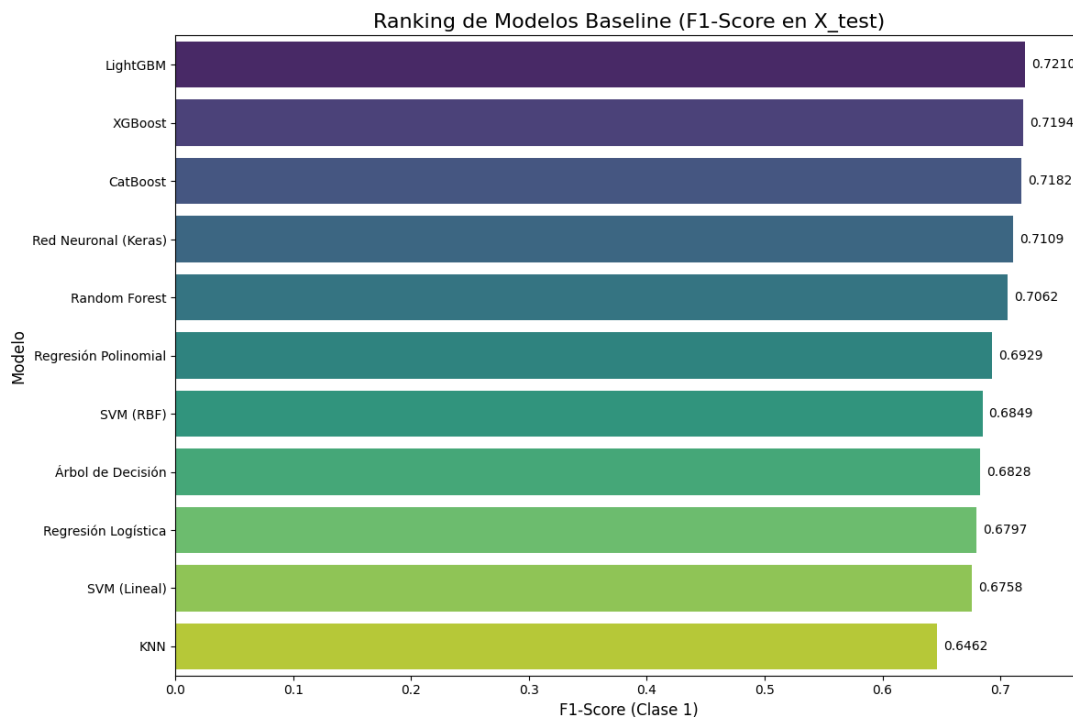


Figura 5: Ranking de Modelos Baseline por F1-Score (Clase 1) en X_{test} .

Conclusión: La búsqueda *baseline* ha identificado a los modelos de Boosting como la familia de algoritmos ganadora. El siguiente paso lógico es, por tanto, realizar una **optimización exhaustiva (GridSearchCV)** únicamente sobre estos tres modelos.

5.5. Optimización Exhaustiva (GridSearchCV)

Con los tres modelos ganadores identificados, se definieron conjuntos más grandes de hiperparámetros. A diferencia de antes(usábamos `RandomizedSearch`), ahora utilizaremos `GridSearchCV` (exhaustiva), el cual prueba sistemáticamente todas las combinaciones. Se definieron rangos amplios, por ejemplo, para `XGBoost`:

```
param_grid_xgb_heavy = {
    'n_estimators': [200, 400, 600, 800],
    'learning_rate': [0.01, 0.03, 0.05, 0.1],
    'max_depth': [3, 5, 7, 9],
    'subsample': [0.7, 0.9, 1.0],
    ...
}
```

Esta fue la parte de la práctica de mayor coste computacional.

5.6. Conclusión y Modelo Final

Los resultados de la optimización exhaustiva, evaluados en `X_test`, se resumen en la Tabla 2.

Cuadro 2: Resultados de F1-Score (Clase 1) tras Optimización Exhaustiva en `X_test`.

Modelo	F1 Baseline	F1 Optimizado (GridSearch)
XGBoost	0.7194	0.7230
LightGBM	0.7210	0.7215
CatBoost	0.7182	0.7185

Tras la búsqueda intensiva, **XGBoost resultó ser el modelo ganador final**, alcanzando un F1-Score de **0.7230** con los hiperparámetros óptimos encontrados (`max_depth: 9`, `n_estimators: 800`, `learning_rate: 0.01`, etc.).

Sin embargo, la observación más relevante es la **mejora marginal** obtenida tras 9.5 horas de optimización (XGBoost mejoró solo +0.0036). Esto sugiere que la búsqueda *baseline* ya había encontrado una combinación de parámetros muy cercana a la óptima.

Interpretamos esto como una indicación de que hemos alcanzado el '**techo**' de rendimiento ($F1 \approx 0.72$) obtenible con las 24 características seleccionadas en el pipeline V2. Como se observa en la matriz de confusión final (Figura 6), el modelo optimizado logra un excelente equilibrio:

- **Recall (Clase 1):** $1178 / (1178 + 164) = 0.87$. Identifica correctamente al 87 % de las personas con salarios altos.

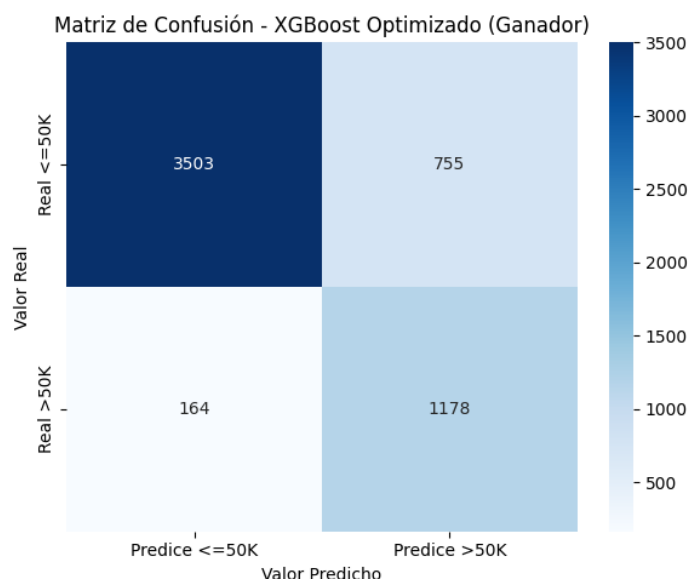


Figura 6: Matriz de Confusión del modelo ganador final (XGBoost Optimizado) en X_{test} .

- **Precisión (Clase 1):** $1178 / (1178 + 755) = 0.61$. El 61 % de sus predicciones positivas son correctas.

En definitiva, este F1-Score de 0.7230 representa un equilibrio robusto entre ambos objetivos.

6. Predicciones Finales

Tras la experimentación y validación de 9 familias de modelos en el apartado anterior, en esta sección hacemos las predicciones reales. Esto consistió en entrenar el modelo ganador seleccionado sobre el *conjunto total de entrenamiento* y generar las predicciones para el *conjunto de prueba* de la competición (Kaggle).

6.1. Carga y Preparación de Datos

A diferencia de la fase de experimentación (que usaba un *split* 80/20 de `salario.csv`), esta fase utilizó los dos archivos proporcionados por la competición:

- **Datos de Entrenamiento:** El archivo completo `salario.csv` (27,998 filas).
- **Datos de Prueba:** El archivo `salario_test.csv` (4,563 filas).

El primer paso fue cargar ambos datasets. En el conjunto de entrenamiento, la variable objetivo `salario` se mapeó a valores numéricos ('<=50K': 0, '>50K': 1). En el conjunto de prueba, la columna ID se extrajo y almacenó para la generación del archivo de envío final.

6.2. Replicación del Pipeline V2

Para asegurar la coherencia, replicamos paso a paso la estrategia de preprocesado ganadora (Pipeline V2) del preprocesado sobre los nuevos datos.

6.2.1. Imputación de Valores Nulos

Ambos conjuntos de datos contenían valores nulos ('?') en las columnas categóricas (**trabajo**, **trabajo.1**, **pais-origen**). Para evitar cualquier fuga de datos, la moda de estas columnas se calculó **únicamente a partir del conjunto de entrenamiento** (**X_train_raw**). Luego, esta moda se utilizó para rellenar los valores nulos tanto en **X_train_raw** como en **X_test_raw**.

6.2.2. Transformación y Selección de Características

Una vez imputados los datos, se aplicó el pipeline V2 completo:

1. **Transformación:** Se instanció el objeto **ColumnTransformer** del Pipeline V2 (con **StandardScaler**, **OrdinalEncoder** y **OneHotEncoder**). Se aplicó **fit_transform()** en **X_train_raw** y solo **transform()** en **X_test_raw**. Esto generó dos conjuntos de datos con 87 características.
2. **Selección (Consenso V2):** Se repitió el proceso de selección de características (Filtro, Wrapper, Embebido) sobre el **X_train_processed** (87 *features*) para recalcular la máscara de '**Consenso ≥ 2** '.
3. **Filtrado Final:** Esta máscara de consenso (que resultó en 24 características) se aplicó tanto a **X_train_processed** como a **X_test_processed**, dejando los datos listos para el modelado con las 24 features finales.

6.3. Selección del Modelo Final

En el apartado anterior realizamos una optimización exhaustiva donde **XGBoost** obtuvo el mejor F1-Score (0.7230) en nuestro *conjunto de validación local*.

Sin embargo, al enviar las predicciones de los modelos optimizados (XGBoost, LGBM y Keras) a la competición de Kaggle, el modelo de **Red Neuronal** demostró una capacidad de generalización superior, alcanzando un F1-Score público de **0.861**.

Dado que el objetivo de este apartado es entrenar el modelo que mejor rendimiento obtenga en datos desconocidos, se seleccionó la Red Neuronal como el modelo definitivo para la predicción real, a pesar de nuestra decisión en el apartado anterior.

6.4. Entrenamiento y Generación de Predicciones

Se procedió a entrenar el modelo Keras seleccionado sobre el 100 % de los datos de entrenamiento ya procesados.

- **Arquitectura y Entrenamiento:** Se utilizó la misma arquitectura de 'embudo' ('Input(24) ->Dense(64) ->Dense(32) ->Dense(1)') y los hiperparámetros de la fase de experimentación (Adam, 50 *epochs*, `batch_size=64` y `fit__class_weight='balanced'`).
- **Predicción y Umbral:** Calculamos las probabilidades en el `X_test` procesado. A estas probabilidades se les aplicó el **umbral de decisión óptimo de 0.69**, identificado en el apartado anterior.
- **Generación de Archivo:** Finalmente, las predicciones binarias (0 y 1) se mapearon de nuevo a sus etiquetas de texto ('<=50K', '>50K'). Estas etiquetas se unieron a los `submission_ids` extraídos al inicio y se guardaron en el archivo `test_labels.csv`